



**UNIVERSIDADE PRESBITERIANA
MACKENZIE Faculdade de
Computação e Informática Laboratório
de Engenharia de Software**

**FOODLINK: SISTEMA DE DOAÇÃO
DE ALIMENTOS Documento de
Especificação e Modelagem**

Integrantes:

- Enzo Ponte Gamberi - RA: 10389931
- João Guilherme Messias de Oliveira Santos – RA: 10426110
- Thiago Ruiz Fernandes Silva – RA: 10426057

Professor: Luiz Carlos Machi Lozano

São Paulo, 2026



SUMÁRIO

1. Introdução
2. Definição da demanda
 - O problema ou oportunidade percebida
 - A razão ou justificativa para esta demanda
 - Descrição sucinta do produto de software
 - Clientes, usuários e demais envolvidos/impactados
 - Principais etapas necessárias para construir o produto
 - Principais critérios de qualidade para o produto
3. Requisitos do Produto
4. Wireframes
5. Modelagem
 - Diagrama de Casos de Uso
 - Diagrama de Domínio/Classe
 - Diagrama de Sequência
6. Descrição da Arquitetura do Sistema e Ferramentas
 - Camada de Front-End
 - Camada de Back-End
 - Banco de Dados



- Diagrama de Implantação
- Ferramentas de Desenvolvimento e DevOps
- Infraestrutura em Nuvem (AWS)

1. INTRODUÇÃO

Entregar um produto de software no prazo, respeitando o orçamento e garantindo alta qualidade são os objetivos fundamentais da Engenharia de Software. Para alcançar essas metas, é imprescindível a aplicação de boas práticas, padrões arquiteturais e técnicas de modelagem que mitigam os riscos de um desenvolvimento inadequado. O presente projeto, desenvolvido no âmbito da disciplina de Laboratório de Engenharia de Software, visa colocar em prática esses recursos metodológicos por meio da proposição e construção de um sistema transacional completo e funcional.

O escopo escolhido para este desafio é o desenvolvimento do **FoodLink**, uma plataforma digital orientada à web com forte caráter extensionista, projetada para solucionar um problema de cenário real. O cenário atual apresenta um paradoxo logístico crítico: altos índices de desperdício de alimentos viáveis para consumo por parte do setor de comércio e serviços, em contraste com a severa insegurança alimentar enfrentada por parcelas vulneráveis da população.

Dessa forma, o FoodLink atua como uma ponte tecnológica, baseada em uma arquitetura *Client-Server*, que estabelece uma rede de comunicação ágil e rastreável entre doadores (pessoas físicas ou empresas) e instituições receptoras (ONGs). Para garantir a integridade das operações — como a prevenção de concorrência simultânea na reserva de um mesmo alimento —, a plataforma adota práticas robustas de modelagem relacional de dados e desenvolvimento back-end através de uma API RESTful.

Este documento atuará como um artefato vivo que evoluirá ao longo de todo o ciclo de vida do software. Nas próximas seções, serão detalhadas as etapas de engenharia de requisitos, a modelagem UML (casos de uso, domínio e sequência), a especificação da arquitetura de software e as diretrizes para a futura implantação automatizada (CI/CD) em ambiente de computação em nuvem, refletindo de forma progressiva a maturidade técnica do projeto.



2. DEFINIÇÃO DA DEMANDA

2.1 O problema ou oportunidade percebida

A raiz do problema que este software se propõe a resolver reside na extrema ineficiência logística e na ausência de canais de comunicação centralizados para doações de perecíveis e não-perecíveis. Diariamente, estabelecimentos comerciais (como padarias, restaurantes e mercados) e pessoas físicas descartam volumes significativos de alimentos em perfeitas condições sanitárias. O descarte ocorre, em grande parte, porque não há um método ágil e confiável para anunciar essas sobras e repassá-las a quem necessita antes de seu vencimento. Além do impacto humano, esse desperdício gera um grave problema ambiental, visto que a decomposição de matéria orgânica é uma das principais fontes de gases de efeito estufa.

2.2 A razão ou justificativa para esta demanda

A justificativa para a construção desta plataforma apoia-se em dois pilares: responsabilidade socioambiental e maturidade tecnológica. No âmbito social, uma plataforma digital dedicada elimina o esforço braçal de organizações não governamentais (ONGs), que gastam recursos escassos buscando doações de forma desestruturada. O sistema mitigará o risco de viagens logísticas perdidas, garantindo o escoamento rápido dos perecíveis. Do ponto de vista técnico e acadêmico, o projeto apresenta desafios didaticamente ricos para a Engenharia de Software: controle de concorrência em transações simultâneas, autenticação estruturada, rastreabilidade de dados e implantação em infraestrutura de nuvem com esteira de CI/CD.

2.3 Descrição sucinta do produto de software

O produto final, denominado **FoodLink**, será uma aplicação web baseada em uma arquitetura *Client-Server*. O sistema gerenciará dois perfis principais de acesso: Doadores e Instituições. O módulo do Doador permitirá o cadastro rápido de alimentos disponíveis. O módulo da Instituição fornecerá um *feed* interativo, com capacidades de filtragem dinâmica, além de um mecanismo transacional rigoroso para reserva exclusiva da doação. O ciclo de vida de um alimento percorrerá os status "Disponível", "Reservado" e "Entregue" (ou "Cancelado"), bloqueando-o para outros usuários e atualizando a interface em tempo real.



2.4 Clientes, usuários e demais envolvidos/impactados

- Usuários Doadores: Pessoas físicas da comunidade dispostas a doar excedentes, bem como entidades jurídicas de pequeno a médio porte (padarias, hortifrutis e restaurantes locais).
- Usuários Receptores (Instituições): Organizações do terceiro setor devidamente cadastradas, abrigos, associações de moradores e frentes de assistência social.
- Impactados Indiretos: A comunidade em situação de vulnerabilidade, que será a beneficiária final dos alimentos arrecadados.

2.5 Principais etapas necessárias para construir o produto

1. Levantamento de Requisitos e Modelagem: Definição do escopo e modelagem UML aprimorada (diagramas de domínio, casos de uso e sequência).
2. Prototipação (Wireframes): Elaboração de protótipos para validar a interface e a usabilidade.
3. Setup de Infraestrutura: Configuração do repositório no GitHub e criação da esteira automatizada de Integração e Entrega Contínua (CI/CD) via GitHub Actions.
4. Desenvolvimento do Back-end: Construção da API RESTful utilizando Python (FastAPI) e modelagem relacional persistida em banco de dados SQLite via ORM.
5. Desenvolvimento do Front-end: Construção de interfaces responsivas com HTML5, CSS3 (Bootstrap) e JavaScript nativo para consumo assíncrono da API.
6. Qualidade e Deploy: Testes de concorrência e implantação definitiva da aplicação em uma instância de nuvem na Amazon Web Services (AWS EC2).

2.6 Principais critérios de qualidade para o produto

- Usabilidade: Interface web simples, limpa e responsiva (adaptável a telas de *smartphones*), permitindo que fluxos críticos ocorram com o mínimo de cliques.
- Confiabilidade e Segurança: Implementação de autenticação baseada em *tokens* (JWT) e proteção de rotas. O banco de dados deve garantir *atomicidade* na operação de reserva, aplicando *locks* (bloqueios de linha) para evitar falhas de concorrência e reservas duplicadas do mesmo item.



- Escalabilidade e Manutenção: Código modular, com separação clara de responsabilidades entre interface e regras de negócio, facilitando a adição futura de novos módulos.
-

3. REQUISITOS DO PRODUTO

A tabela a seguir consolida o backlog do produto, classificando os requisitos identificados em Funcionais (RF) e Não Funcionais (RNF), ordenados por prioridade.

ID	Tipo	Descrição Técnica do Requisito	Prioridade
RF01	RF	O sistema deve permitir o cadastro de contas de usuário, diferenciando o perfil entre "Doador" e "Instituição".	Alta
RF02	RF	O sistema deve possuir um mecanismo de autenticação (Login/Logout) validando as credenciais de forma segura.	Alta
RF03	RF	O Doador deve poder registrar um alimento, informando nome, quantidade, data de validade e categoria (ex: perecível, não-perecível).	Alta
RF04	RF	A Instituição deve poder visualizar um "feed" contendo todas as doações exclusivas com o status "Disponível".	Alta
RF05	RF	A Instituição deve poder solicitar a reserva de um alimento, o que alterará o status da doação para "Reservado" e exibirá os dados do doador para retirada.	Alta
RF06	RF	O doador deve poder atualizar o status da doação para "Entregue" após a efetivação física da retirada pela Instituição.	Alta



RF07	RF	O sistema deve prover um filtro de pesquisa dinâmico para que a Instituição busque alimentos por categorias específicas no feed.	Média
RF08	RF	O sistema deve permitir que o doador ou a instituição cancele a operação (doação ou reserva) caso ocorra um imprevisto logístico.	Média
RF09	RF	O sistema deve permitir que o usuário gerencie seu perfil, alterando dados cadastrais como endereço ou senha.	Baixa
RNF01	RNF	O sistema deve ser responsivo e acessível, com a interface do usuário adaptando-se corretamente a navegadores desktop e dispositivos móveis (telas a partir de 375px de largura).	Alta
RNF02	RNF	O sistema deve garantir a segurança dos dados armazenando as senhas dos usuários exclusivamente através de funções de <i>hash</i> criptográfico (ex: bcrypt), sendo vedado o armazenamento em texto puro no banco de dados.	Alta
RNF03	RNF	O <i>backend</i> da aplicação (API RESTful) deve processar as reservas de alimentos garantindo a <i>atomicidade</i> da transação no banco de dados, prevenindo falhas de concorrência onde duas Instituições reservem o mesmo item simultaneamente.	Média
RNF04	RNF	A plataforma deve ser implantada e hospedada em infraestrutura de nuvem (AWS EC2), garantindo disponibilidade contínua para acesso público via internet.	Média



4. WIREFRAMES (PROTÓTIPOS DE BAIXA FIDELIDADE)

TELA DE LOGIN

Sistema de Doação

Email

Senha

☐ Doador ☐ Instituição

Tela da Instituição (Feed de Alimentos)

Buscar:

ALIMENTOS DISPONÍVEIS:

Ex: 5Kg de Feijão - Mercado XYZ
Validade: 30/03/2026

EX: 30 Pães Franceses - Padaria ABC
Validade: Hoje



Tela do Doador (Cadastrar Alimento)

Nova Doação

Quantidade:

Ex: 20
unid

Validade:

Ex:
25/03/2026

Categoria:

Selecione... v

Cadastro de Novo Usuário

Criar Nova Conta

Nome:

CPF/CNPJ:

Endereço:

Email:

Senha:

☐ Sou Doador ☐ Sou Instituição

Painel / Histórico

Meu Histórico:

- 5kg de Arroz - Entregue dia 10/03
- 20 Pães - Reservado - Aguardando Retirada
- 10 Maças - Ainda Não Reservada



A elaboração de wireframes de baixa fidelidade tem o objetivo de validar a arquitetura da informação visual de forma rápida e com baixo custo. O foco nesta etapa é garantir que os fluxos de navegação propostos atendem aos critérios de usabilidade antes do início da codificação das interfaces. Para mapear o fluxo completo do sistema, foram projetadas as seguintes telas principais:

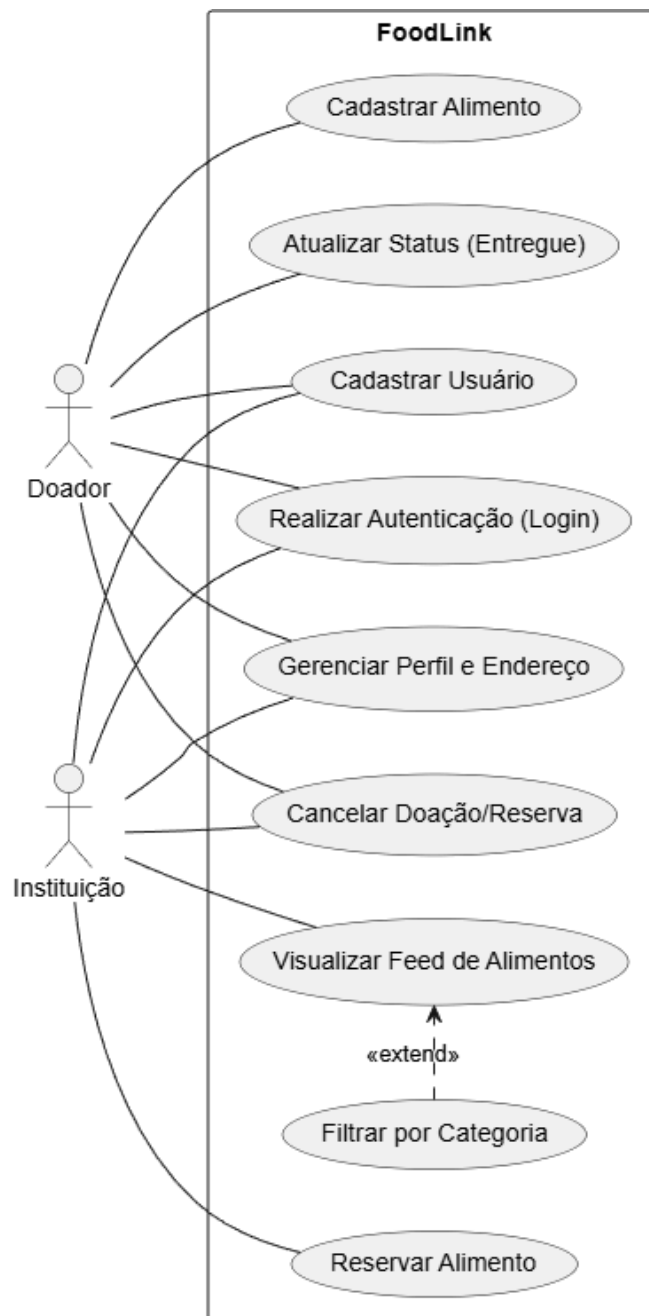
1. **Interface de Autenticação (Login):** Ponto de entrada do sistema, contendo os campos de credenciais (e-mail e senha) e os seletores de perfil para direcionamento correto (Doador ou Instituição).
2. **Interface de Cadastro de Novo Usuário:** Formulário unificado para a entrada de novos atores no sistema, coletando dados essenciais de contato e endereço para viabilizar as futuras coletas logísticas.
3. **Dashboard do Doador (Cadastro de Alimento):** Painel focado na conversão rápida, permitindo que o doador registre um item preenchendo apenas as informações vitais da regra de negócio (nome, quantidade, validade e categoria).
4. **Feed da Instituição (Busca e Reserva):** Tela central da Instituição, projetada no formato de cartões (*cards*) para fácil leitura. Inclui uma barra superior de pesquisa para filtragem ágil e acesso direto ao botão de reserva.
5. **Painel de Histórico e Acompanhamento:** Interface de controle que permite aos usuários rastrear o ciclo de vida das transações no sistema, diferenciando itens concluídos (entregues), em andamento (reservados) e pendentes.

5. MODELAGEM LEVE DO SISTEMA

A modelagem do sistema tem o papel de abstrair a complexidade do software utilizando diagramas UML, padronizando o entendimento das regras de negócio e estruturando o escopo de desenvolvimento técnico para toda a equipe. Nesta etapa, o projeto **FoodLink** foca na correção das associações de atores, no detalhamento rigoroso dos fluxos e na expansão do modelo de dados para garantir a escalabilidade prometida.

5.1 Diagrama de Casos de Uso

O diagrama de casos de uso mapeia as funcionalidades do sistema sob a ótica dos usuários (Atores). Seguindo as normas da UML, as associações entre os atores (Doador e Instituição) e os casos de uso são representadas por linhas sólidas sem pontas de seta, indicando participação na funcionalidade sem confusão com fluxos de dados ou herança.





Especificação dos Casos de Uso

A fim de guiar o desenvolvimento do back-end e garantir a correta implementação das regras de negócio, os casos de uso mapeados no diagrama acima são detalhados a seguir:

- **UC01 – Cadastrar Usuário**
 - **Ator Principal:** Novo Usuário (Doador ou Instituição).
 - **Atores Secundários:** Banco de Dados.
 - **Pré-condições:** Nenhuma.
 - **Pós-condições:** Usuário cadastrado e senha armazenada com segurança.
 - **Fluxo Principal:**
 - O usuário acessa a página de cadastro.
 - O usuário preenche Nome, E-mail, Senha, Endereço e seleciona o tipo de perfil (Doador ou Instituição).
 - O sistema valida se o e-mail já existe.
 - O sistema criptografa a senha e salva os dados no banco.
 - O sistema exibe mensagem de sucesso.
 - **Fluxo de Exceção:** E-mail já cadastrado. O sistema avisa o usuário e solicita um novo e-mail.
- **UC02 – Realizar Autenticação (Login)**
 - **Ator Principal:** Usuário (Doador ou Instituição).
 - **Atores Secundários:** Banco de Dados.
 - **Pré-condições:** Possuir cadastro no sistema.
 - **Pós-condições:** Sessão iniciada com Token JWT gerado.
 - **Fluxo Principal:**
 - O usuário insere e-mail e senha.
 - O sistema verifica as credenciais no banco de dados.
 - O sistema gera um token de acesso.
 - O sistema redireciona o usuário para o painel correspondente ao seu perfil.
 - **Fluxo de Exceção:** Credenciais incorretas. O sistema avisa que o e-mail ou senha estão errados.
- **UC03 – Gerenciar Perfil e Endereço**
 - **Ator Principal:** Usuário Autenticado.
 - **Atores Secundários:** Banco de Dados.
 - **Pré-condições:** Usuário deve estar logado.
 - **Pós-condições:** Dados atualizados com sucesso.
 - **Fluxo Principal:**
 - O usuário acessa as configurações de perfil.
 - O usuário altera os dados (nome, senha ou endereço).
 - O sistema salva as alterações no banco de dados.



- O sistema confirma a atualização.

- **UC05 – Atualizar Status (Entregue)**

- **Ator Principal:** Doador.
- **Atores Secundários:** Banco de Dados.
- **Pré-condições:** O alimento deve estar com status "Reservado".
- **Pós-condições:** Status alterado para "Entregue".
- **Fluxo Principal:**
 - O Doador acessa suas doações.
 - O Doador confirma que a Instituição retirou o alimento fisicamente.
 - O sistema altera o status para "Entregue".
 - O alimento sai do fluxo ativo.

- **UC06 – Cancelar Doação/Reserva**

- **Ator Principal:** Doador ou Instituição.
- **Atores Secundários:** Banco de Dados.
- **Pré-condições:** O item não pode estar com status "Entregue".
- **Pós-condições:** Operação cancelada.
- **Fluxo Principal:**
 - O usuário clica em cancelar na doação ou reserva ativa.
 - O sistema solicita confirmação.
 - Se for cancelamento de doação (Doador), o item é removido.
 - Se for cancelamento de reserva (Instituição), o item volta a ficar "Disponível".

- **UC07 – Visualizar Feed de Alimentos**

- **Ator Principal:** Instituição.
- **Atores Secundários:** Banco de Dados.
- **Pré-condições:** Usuário logado com perfil de Instituição.
- **Pós-condições:** Lista de alimentos disponíveis exibida.
- **Fluxo Principal:**
 - A Instituição acessa a tela principal.
 - O sistema busca no banco todos os itens com status "Disponível".
 - O sistema exibe os itens em formato de cartões.

- **UC08 – Filtrar por Categoria**

- **Ator Principal:** Instituição.
- **Atores Secundários:** Banco de Dados.
- **Pré-condições:** Estar na tela de Feed.
- **Pós-condições:** Feed atualizado com o filtro aplicado.
- **Fluxo Principal:**
 - A Instituição seleciona uma categoria (ex: "Perecíveis").
 - O sistema busca apenas itens daquela categoria.



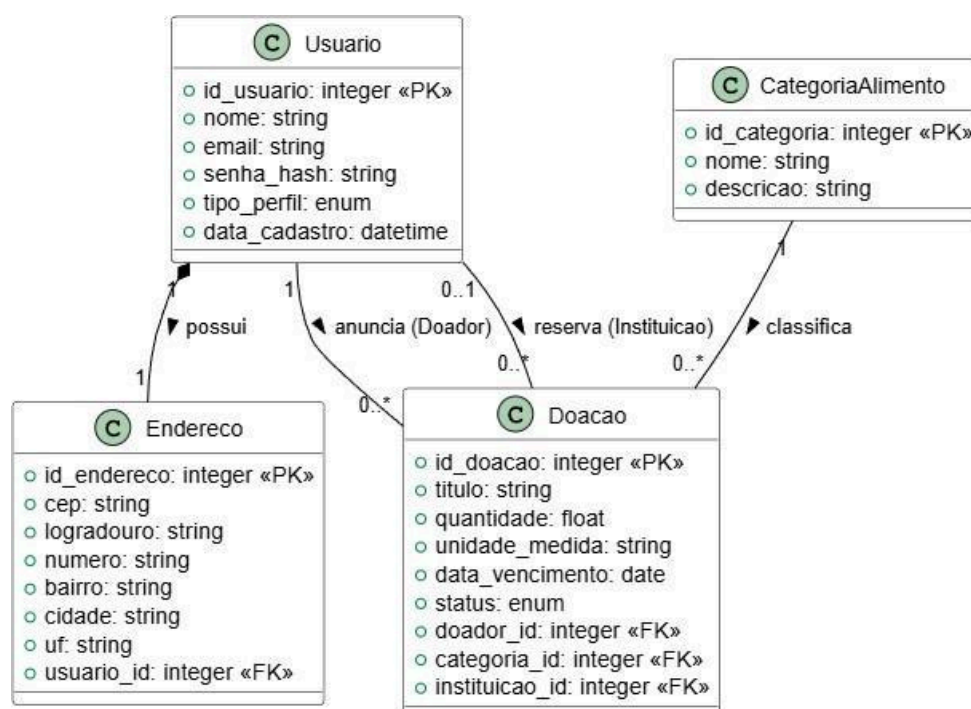
- O sistema atualiza a tela com o resultado.

- **UC09 – Reservar Alimento**

- **Ator Principal:** Instituição.
- **Atores Secundários:** Banco de Dados.
- **Pré-condições:** Item deve estar "Disponível".
- **Pós-condições:** Status alterado para "Reservado" e endereço de retirada liberado.
- **Fluxo Principal:**
 - A Instituição clica em "Reservar" no alimento escolhido.
 - O sistema valida se o item ainda está disponível.
 - O sistema altera o status para "Reservado" e associa à Instituição.
 - O sistema mostra o endereço do Doador para a Instituição.
- **Fluxo de Exceção:** Item já reservado por outra pessoa. O sistema avisa que o item não está mais disponível.

5.2 Diagrama de Domínio/Classe

O diagrama de domínio detalha as entidades estruturais fundamentais que serão persistidas no banco de dados e as multiplicidades de suas relações lógicas. Atendendo ao feedback da primeira etapa, o modelo foi expandido para contemplar a complexidade real do sistema **FoodLink**, passando de uma estrutura simplista para um modelo normalizado com quatro entidades principais e relações bem definidas.





Especificação das Entidades e Atributos

A modelagem de dados foi estruturada para garantir rastreabilidade, segurança e flexibilidade no gerenciamento das doações:

- **Entidade Usuario:** Centraliza a autenticação e o perfil dos atores.
 - **Atributos:** id_usuario (PK - int), nome (string), email (string - único), senha_hash (string), tipo_perfil (enum: DOADOR, INSTITUICAO), data_cadastro (datetime).
- **Entidade Endereco:** Classe separada para normalização, vinculada ao usuário por uma relação de composição. Isso facilita a logística de retirada.
 - **Atributos:** id_endereco (PK - int), cep (string), logradouro (string), numero (string), bairro (string), cidade (string), uf (string), usuario_id (FK).
- **Entidade Doacao:** Representa o item anunciado. É a classe central das regras de negócio.
 - **Atributos:** id_doacao (PK - int), titulo (string), quantidade (float), unidade_medida (string), data_vencimento (date), status (enum: DISPONIVEL, RESERVADO, ENTREGUE, CANCELADO), doador_id (FK), categoria_id (FK), instituicao_id (FK - opcional).
- **Entidade CategoriaAlimento:** Permite a classificação dinâmica dos alimentos (ex: Perecíveis, Hortifrúti, Grãos), alimentando os filtros de busca.
 - **Atributos:** id_categoria (PK - int), nome (string), descricao (string).
- **Multiplicidades (Regras de Negócio):**
 - Possui (1 para 1): Cada usuário possui exatamente um Endereço. A relação é de composição, indicando que o endereço logístico é dependente da existência do usuário no sistema.
 - Anuncia (1 para 0..*): Um usuário (do tipo Doador) pode anunciar zero ou várias doações, mas cada doação pertence a apenas um doador.
 - Reserva (0..1 para 0..*): Uma Doação pode ser reservada por, no máximo, uma Instituição por vez, garantindo a exclusividade e evitando conflitos logísticos. Uma instituição pode possuir múltiplas reservas ativas.
 - Classifica (1 para 0..*): Uma CategoriaAlimento pode agrupar diversas



UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática



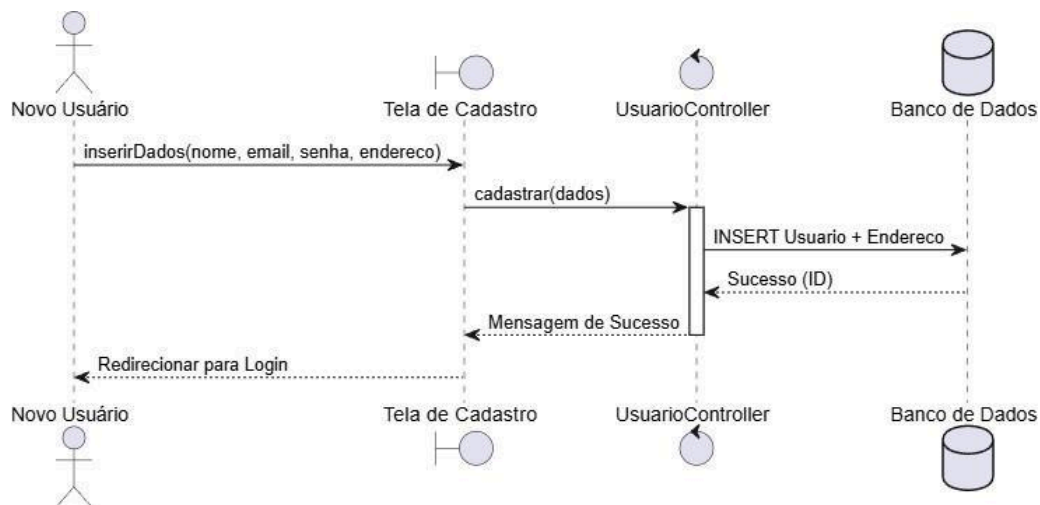
Doações, mas cada doação deve obrigatoriamente pertencer a uma

categoria cadastrada para facilitar a filtragem no *feed*.

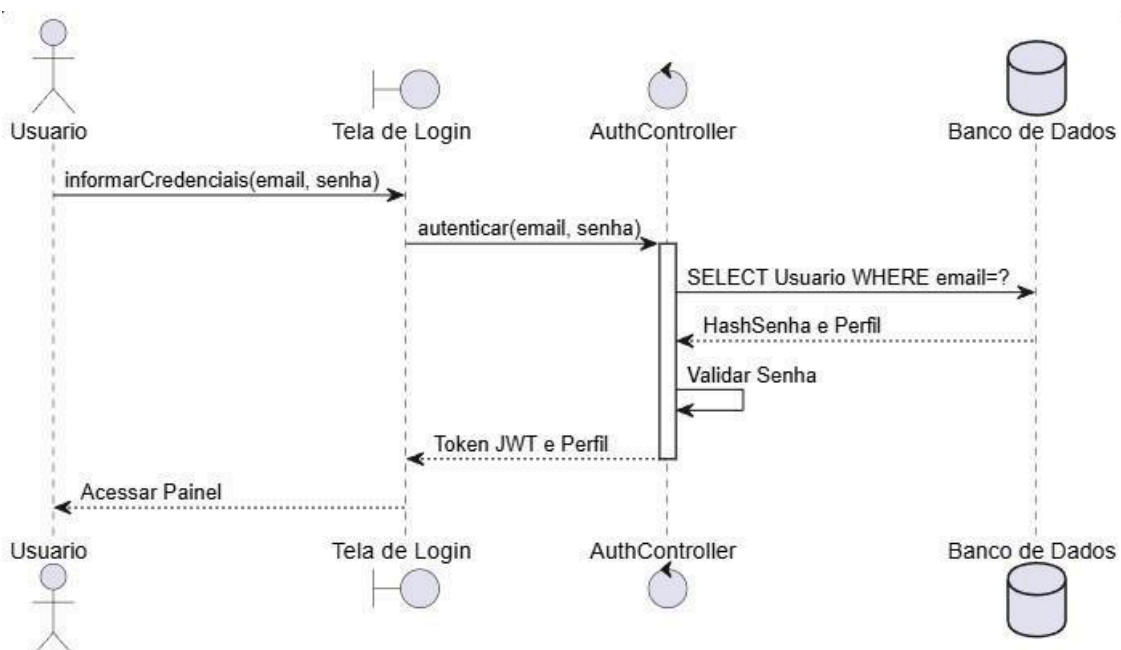
5.3 Diagrama de Sequência

Os diagramas de sequência abaixo descrevem a interação detalhada entre os objetos do sistema ao longo do tempo para cada funcionalidade principal do **FoodLink**. Essa modelagem é essencial para orientar a implementação do *back-end* (API) e do *front-end*, garantindo que as regras de negócio sejam respeitadas em cada transação

UC01 - Cadastrar Usuário

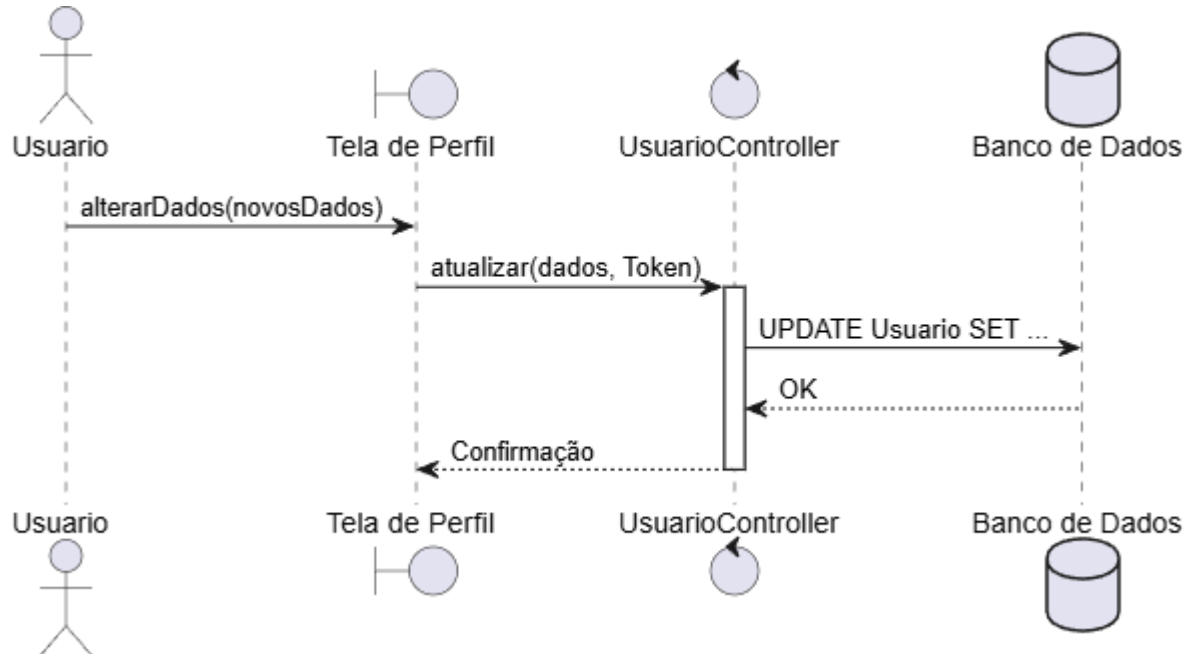


UC02 - Realizar Autenticação (Login)

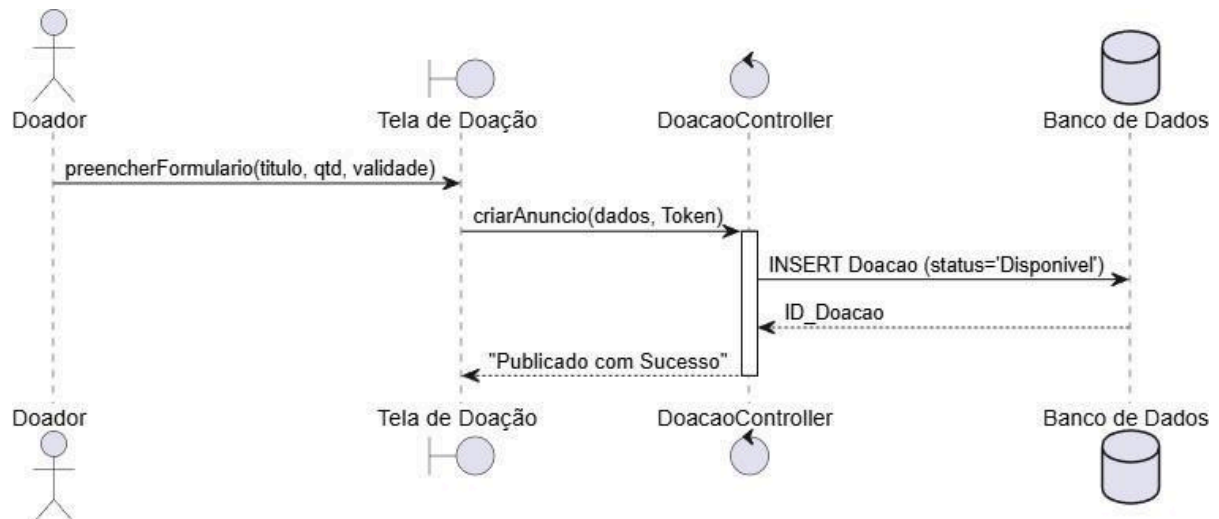




UC03 - Gerenciar Perfil e Endereço

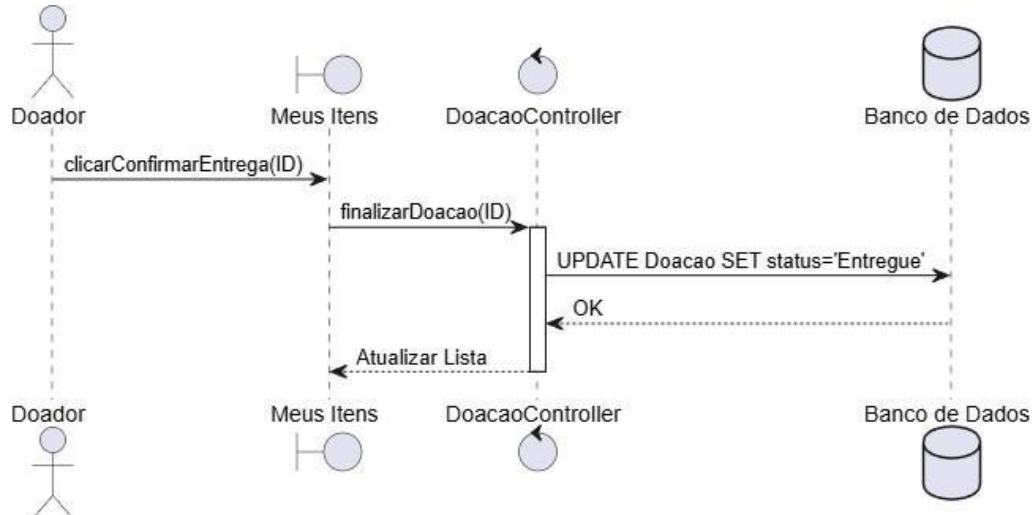


UC04 - Cadastrar Alimento

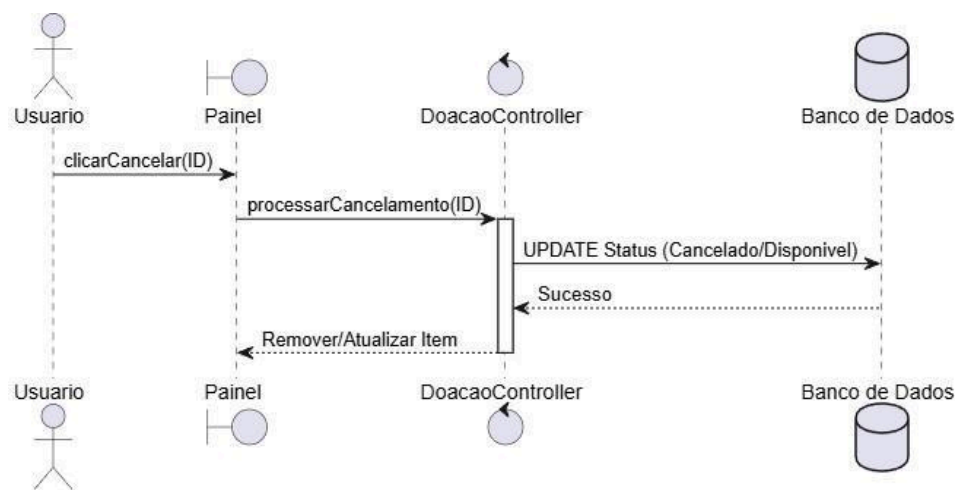




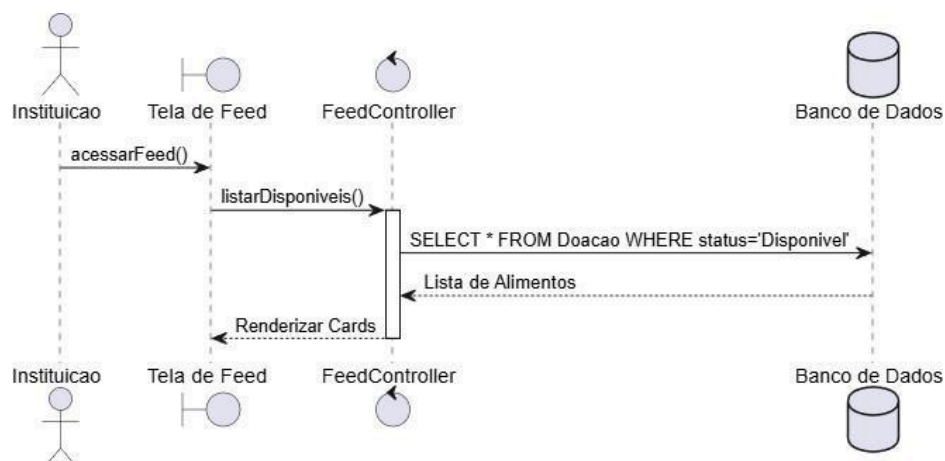
UC05 - Atualizar Status (Entregue)



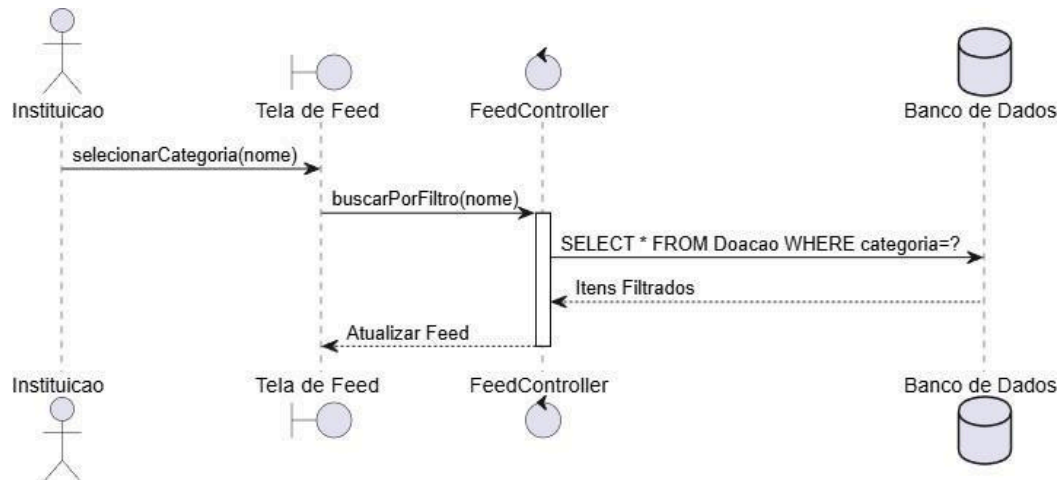
UC06 - Cancelar Doação/Reserva



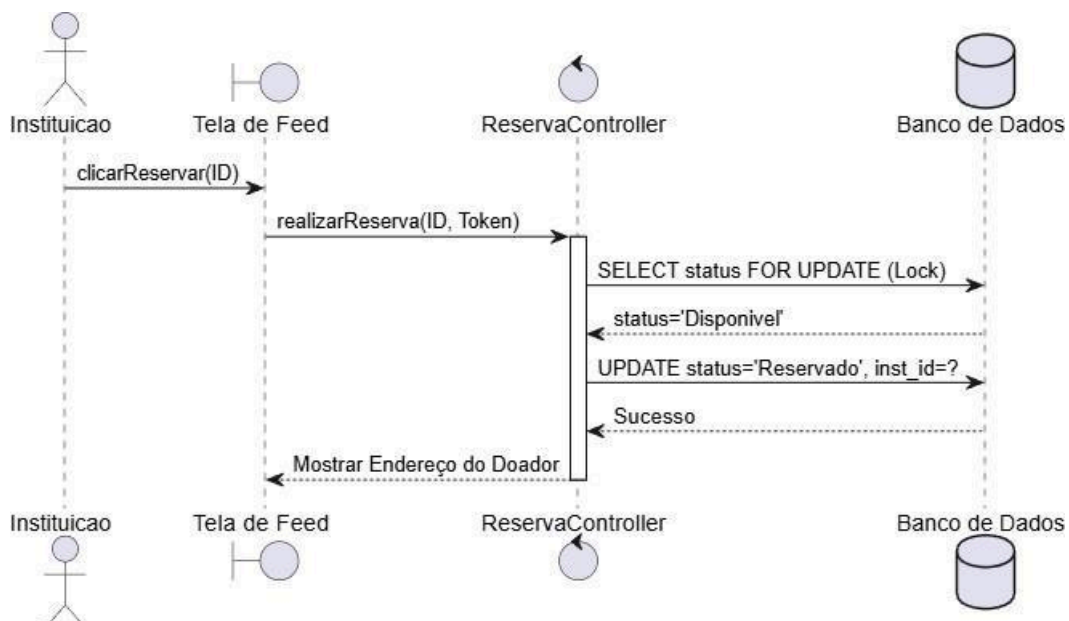
UC07 - Visualizar Feed de Alimentos



UC08 - Filtrar por Categoria



UC09 - Reservar Alimento (Com Lock)





6. DESCRIÇÃO DA ARQUITETURA DO SISTEMA E FERRAMENTAS

Para atender de forma estrita aos requisitos obrigatórios do projeto — separação de *front-end* e *back-end*, consumo de API e hospedagem em nuvem — o **FoodLink** adota uma arquitetura moderna baseada em serviços, utilizando o modelo *Client-Server*. Essa abordagem garante que a interface do usuário e as regras de negócio evoluam de forma independente, facilitando a manutenção e a escalabilidade do sistema.

6.1 Camada de Front-end (Interface do Usuário)

O front-end é desenvolvido com tecnologias nativas da web: **HTML5**, **CSS3** e **JavaScript (Vanilla)** puro.

- **Justificativa:** Optou-se por não utilizar frameworks pesados (como React) para manter o foco na lógica de integração e evitar complexidades desnecessárias na esteira de build. O JavaScript assíncrono, via API nativa `fetch()`, é responsável pelas chamadas HTTP para o back-end e pela injeção dinâmica de dados na tela.
- **Estilização:** Utiliza-se o framework **Bootstrap**, que garante uma interface responsiva (adaptável a celulares e desktops) e uma usabilidade padronizada.

6.2 Camada de Back-end (API e Regras de Negócio)

A API RESTful do sistema é desenvolvida em **Python**, utilizando o framework **FastAPI**.

- **Justificativa:** O FastAPI foi escolhido por ser um microframework de alta performance que oferece validação automática de dados e geração de documentação interativa (Swagger). Ele processa requisições em formato JSON enviadas pelo cliente e gerencia a lógica de autenticação e segurança.
- **Segurança:** O sistema utiliza **JWT (JSON Web Tokens)** para gerenciar sessões de forma segura e stateless, garantindo que apenas usuários autenticados acessem rotas críticas.



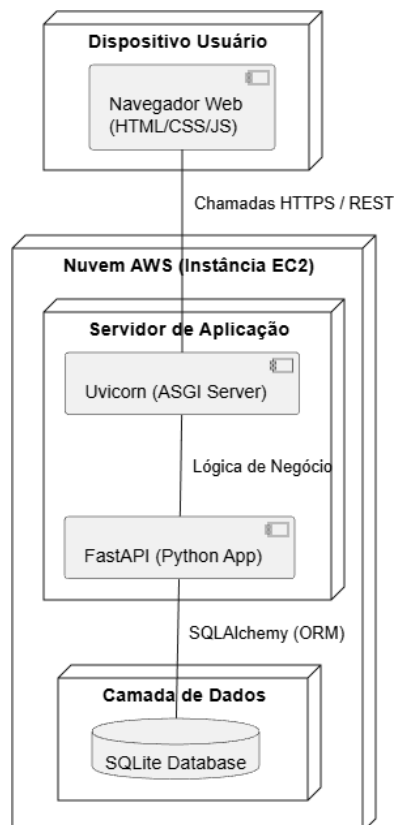
6.3 Camada de Persistência de Dados (Banco de Dados)

O gerenciamento dos dados é feito através da linguagem **SQL**, utilizando o banco de dados **SQLite** para a fase de desenvolvimento e produção inicial.

- **Justificativa:** O SQLite é um banco relacional leve que não requer um servidor externo complexo, sendo ideal para a agilidade do projeto. A interação com o banco é mediada pelo ORM **SQLAlchemy**, que abstrai as consultas SQL e garante a integridade das transações (como a prevenção de reservas duplicadas)

6.4 Diagrama de Implantação (Arquitetura de Hardware e Software)

O diagrama de implantação ilustra a distribuição física dos componentes do sistema, detalhando a comunicação entre o dispositivo do usuário e a infraestrutura na nuvem.





Descrição dos Nós:

1. **Dispositivo do Usuário:** Representa o hardware (PC ou celular) executando o navegador web.
2. **AWS Cloud (Instância EC2):** O servidor virtual que hospeda a aplicação. Contém o servidor **Uvicorn** (que executa a API Python) e os arquivos estáticos do front-end.
3. **Banco de Dados:** O arquivo SQLite persistido no disco da instância EC2.

6.5 Ferramentas de Desenvolvimento e DevOps

- **Editor de Código:** Utilizaremos o **Visual Studio Code (VSCode)** devido à sua leveza e integração nativa com extensões essenciais para Python e versionamento.
- **Controle de Versão:** A base de código será hospedada no **GitHub**, cumprindo o requisito obrigatório de atuar como sistema de gerenciamento de versões em nuvem e facilitando a colaboração simultânea dos membros do grupo.
- **Integração e Entrega Contínua (CI/CD):** A plataforma **GitHub Actions** será utilizada para criar a esteira automatizada. Toda vez que um desenvolvedor enviar (*push*) um novo código para o repositório, um script executará validações automáticas para garantir que a aplicação não contenha erros de sintaxe antes de ser implantada.

6.6 Infraestrutura em Nuvem (AWS)

Para atender às diretrizes de escalabilidade e disponibilidade do projeto **FoodLink**, a aplicação será implantada na infraestrutura de nuvem da **Amazon Web Services (AWS)**. O sistema deixará o ambiente de desenvolvimento local para ser hospedado em uma instância **Amazon EC2 (Elastic Compute Cloud)** com sistema operacional Linux (Ubuntu), configurada para operar como servidor de produção 24 horas por dia.

A arquitetura de implantação na nuvem foi desenhada para garantir segurança e performance, contando com os seguintes componentes técnicos:

- **Servidor de Aplicação (Uvicorn):** Responsável por executar a API Python (FastAPI) de forma assíncrona.
- **Proxy Reverso e Estáticos:** Utilização do **Nginx** para gerenciar o tráfego de rede e servir os arquivos de *Front-end* (HTML/JS) com alta velocidade.



- **Segurança e Criptografia (HTTPS):** Implementação de certificados SSL/TLS (via Let's Encrypt), garantindo que todos os dados trafegados entre o doador/ONG e a AWS sejam criptografados.
- **Persistência Local:** O banco de dados **SQLite** será mantido no volume de armazenamento da instância, garantindo acesso de baixa latência para as operações de reserva e cadastro.

Esta configuração valida o caráter extensionista do projeto, permitindo que a plataforma seja acessada publicamente via internet por qualquer instituição ou doador através de uma URL segura, simulando um ambiente real de mercado.

7. CASOS DE TESTES AUTOMATIZADOS

Para garantir a qualidade do software e a integridade das regras de negócio mapeadas na fase de modelagem, foi implementada uma suíte de testes automatizados. O foco principal desta etapa de testes unitários é a validação do ciclo de vida da entidade Doacao (Disponível → Reservado → Entregue / Cancelado), que é o núcleo transacional do sistema FoodLink.

Os testes foram desenvolvidos utilizando a biblioteca nativa unittest da linguagem Python, assegurando que as transações de reserva e entrega ocorram sem falhas lógicas. A execução contínua destes testes foi automatizada através de uma esteira de CI/CD (Integração e Entrega Contínua) configurada no GitHub Actions.

A tabela a seguir documenta os casos de teste implementados, seus objetivos e os resultados obtidos na execução da esteira:

ID	Método Testado	Descrição do Caso de Teste	Resultado Esperado	Status na Esteira
CT01	<i>test_criacao_status</i>	Cenário A: Verificar o status padrão ao instanciar uma nova doação. Cenário B: Verificar se os atributos (ex: título do alimento) são	A doação deve ser criada obrigatoriamente e com o status inicial "Disponível". Os atributos devem corresponder aos dados de entrada.	APROVADO



		armazenados corretamente na memória.		
CT02	<i>test_reservar</i>	Cenário A: Efetuar a reserva de um item que está com status "Disponível". Cenário B: Tentar efetuar a reserva de um item que já possui o status "Reservado" (prevenção de dupla reserva).	Cenário A: Retorna True e o status muda para "Reservado". Cenário B: Retorna False, impedindo a sobreposição de reservas.	APROVADO
CT03	<i>test_entregar</i>	Cenário A: Tentar confirmar a entrega de um item que ainda está "Disponível" (não reservado). Cenário B: Confirmar a entrega de um item devidamente "Reservado".	Cenário A: Retorna False, pois um item não pode ser entregue sem reserva prévia. Cenário B: Retorna True e o status muda para "Entregue".	APROVADO
CT04	<i>test_cancelar_reserva</i>	Cenário A: Tentar cancelar a reserva de um item que ainda está "Disponível". Cenário B: Cancelar ativamente a reserva de um item com status "Reservado".	Cenário A: Retorna False (operação inválida). Cenário B: Retorna True e o status do item é revertido para "Disponível", retornando ao feed da aplicação.	APROVADO
CT05	<i>test_alterar_quantidade</i>	Cenário A: Atualizar a quantidade da	Cenário A: Retorna True e a quantidade é	APROVADO



		doação informando um valor numérico válido (maior que zero). Cenário B: Tentar atualizar a quantidade informando um valor inválido (zero ou negativo).	atualizada com sucesso. Cenário B: Retorna False, a operação é bloqueada e a quantidade original é preservada.	
--	--	--	--	--